

COMP3023: Design and Analysis of Algorithms

Lecture 14: Computational Complexity Theory

Computer Science and Technology
United International College

- 1 Introduction
 - Motivation
 - Input size
 - Decision vs Optimization
- 2 Problem Classes
 - **P**
 - **NP**
 - **NP-complete**
 - Reduction
 - A small taste on **NP-c** proof
- 3 Complexity Zoo

- We have learned several algorithm design strategies, for example divide-and-conquer, dynamic programming, and greedy algorithms.
- But what if the problem is “too difficult” and none of these strategies finds the correct solution?
- Showing a problem has an efficient algorithm is easy, construct the algorithm.
- It is not easy to show a problem has no efficient algorithm - hard to show something does not exist.
- To classify the difficulty of problems, we study computational complexity theory.
- In this course, we introduce the problem class **P**, **NP**, **co-NP**, **NP-complete**, and **NP-hard**.

Informally, we have defined

- a computational problem is a relation between input and output;
- the running time of an algorithm is a function $T(n)$ of the input size n .

However, the input size sometimes relies on the representation, called *encoding*. For example,

- Prim's algorithm finds the MST in $O(e \log n)$ time for the input graph $G = (V, E)$.
- Computers cannot accept a graph as the input, unless it is represented by an adjacency list or adjacency matrix.
- Suppose we use adjacency matrix. This matrix is an encoding of the graph. The size of the matrix is n^2 .
- Thus, Prim's algorithm also runs in $O(n^2 \log n)$ time.

Definition

Input size(formal) The input size of a problem is the minimum number of bits ($\{0,1\}$) needed to encode the input of the problem.

Remark:

- The above definition encodes the input in binary. Acutally, the encoding can be in an arbitrary-ary system.
- The arity of an encoding system gives different length, but won't affect the asymptotical analysis.
- The input size is hard to compute - undecidable in general (See Kolmogorov complexity for details).

Example

- Graph problems, using the adjacency matrix - input size is v^2 where v is the number of vertices
- Integer factorization, using the binary number system - input size is $\log n$ where n is the number you want to factorize.
- Sorting problem, using binaries in fixed length - $\log \max(a_i)$ for each integer and $n \log \max(a_i)$ in total.

Definition

Two positive functions $f(n)$ and $g(n)$ are of the **same type** if

$$c_1 g(n^{a_1})^{b_1} \leq f(n) \leq c_2 g(n^{a_2})^{b_2}$$

for all large n , where $a_1, b_1, c_1, a_2, b_2, c_2$ are some positive constants.

- For example, all polynomials are of the same type, while polynomials and exponentials are of different types.
- If we consider input encodings are functions from input instances to binary strings, then we are free to talk two encodings are of the same type.
- For example, expressing a graph by an adjacency matrix or an adjacency list are of the same type.

Two types of problems

Definition

A *decision problem* is a question that has only two possible answers, **yes** or **no**.

Definition

And *optimization problem* requires an answer that is an optimal configuration.

For example,

- “Find the shortest path from v to u in graph G ” is an optimization problem;
- “Does graph G have a path from v to u of length shorter than d ” is a decision problem.

Generally, an optimization problem is more difficult than the corresponding decision problem.

Definition

An instance of a decision problem is a *yes-input* if the answer to the instance is yes, *no-input* otherwise.

Definition

Let L be a decision problem. The decision problem $\bar{L}$ is the **complement** of L if every yes-input of $\bar{L}$ is a no-input of L .

For example,

- **Prime**: Is given positive integer n a prime number?
- **Composite**: Is given positive integer n a composite?
- $\text{Prime} = \overline{\text{Composite}}$
- $\text{Composite} = \overline{\text{Prime}}$

Definition (Informal)

The *difficulty* of a computational problem is defined by the complexity of the “best” algorithm which solves the problem.

Definition

A problem is *solvable in polynomial time* if it can be solved by an algorithm in polynomial time.

- For example, sorting, shortest path, ...
- For the sorting problem, definitely you can enumerate all possible sequence and find the sorted one, which takes factorial time. But the “fastest” algorithm runs in polynomial time.

Definition

The class $\mathbf{P}$ consists of all decision problems which are solvable in polynomial time.

- To prove a decision problem in $\mathbf{P}$, you need to construct a polynomial algorithm to solve it.
- To prove a problem not in $\mathbf{P}$, you need to show no polynomial algorithm can solve it, which is much harder.

Usually a decision problem is described as:

Is there an object satisfying some conditions?

Definition

A *certificate* (or *proof*) is a specific object satisfying the conditions.

Definition

Verifying a certificate is a process to check the given object satisfies the condition.

Example

- **Problem:** Is the given positive integer n a composite?
- **Instance:** $n = 341$
- **Certificate:** $c = 31$
- **Verifying the certificate:** $n \bmod c = 341 \bmod 31 = 0$
- Because the certificate is verified, $n = 341$ is a composite.

Note that if the certificate is wrong, the input instance is not necessarily a no-answer. For example, the certificate $c = 2$ to the above instance is wrong. It does not mean 341 is a prime.

The Class **NP**

The class **NP** has two equivalent definitions.

Definition

The class **NP** is the set of decision problems which can be solved by a *nondeterministic turing machine* in polynomial time.

Definition

The class **NP** is the set of decision problems such that for each yes-input, there exists a certificate that can be verified in polynomial time.

- For example, **COMPOSITE** $\in$ **NP** because $n \bmod c$ can be calculated in polynomial time.
- **NP** stands for **nondeterministic polynomial time**. It is **NOT** “non polynomial time”.
- We are not interested in no-inputs.

P=NP?

- One of the most important problems in computer science is **P=NP** or **P $\neq$ NP**?
- In other words, is every problem that can be **verified** in polynomial time also **solvable** (or decidable) in polynomial time?
- Or, are there some polynomially verifiable problems cannot be solved in polynomial time?
- At first glance, it seems “obvious” **P $\neq$ NP**. But the problem remains open for the past 40 years.
- **NP\P** is called **NP-intermediate**, the set of **NP** problems cannot be solved in polynomial time. Here are some candidates:
 - Factoring integers (proved in **P**, 2004),
 - Graph isomorphism problem (quasipolynomial, 2015), and
 - Minimum circuit size problem.

Some **NP** problems seems to be harder than other **NP** problems.

Definition (Informal)

The class **NP-complete** is a set of **NP** problems such that if one is solved in polynomial time, then all **NP** problems can be solved in polynomial time.

Reduction

Let P_1 and P_2 are two decision problems. X_1 and X_2 are the sets of all inputs of P_1 and P_2 respectively. x_1 is an instance of P_1 .

Definition

The *reduction* from P_1 to P_2 is a function $r : X_1 \rightarrow X_2$ such that x_1 is a yes-input for P_1 if and only if $r(x_1)$ is a yes-input for P_2 .

The intuition is if P_1 can be reduced to P_2 and A_2 is an algorithm solving P_2 , then we can construct the following algorithm which solves P_1 .

Algorithm: P_1 solver

Input: An instance x_1

Output: "Yes" if x_1 is a yes-input, "No" otherwise

```
1 begin
2    $x_2 \leftarrow r(x_1)$ ;
3   if  $A_2(x_2)$  (may need other conditions in some cases) then
4     return "Yes";
5   else
6     return "No";
```

Example

Here we give a small example of reductions. Consider the following two problems.

P_1 : Subset-Sum

- Given a set A of n numbers and another number s , does the set A have a subset A' such that $\sum_{x \in A'} x = s$?
- For example, $A = \{1, 2, 3, 4\}$ and $s = 8$. The answer is “yes” because $A' = \{1, 3, 4\}$ and $\sum_{x \in A'} x = 1 + 3 + 4 = 8$.

P_2 : Set-Partition

- Given a set A of n numbers, can the set A be splitted to two subsets A'_1 and A'_2 such that the $\sum_{x \in A'_1} x = \sum_{x \in A'_2} x$?
- For example, $A = \{1, 2, 3\}$. The answer is “yes” because $A'_1 = \{1, 2\}$ and $A'_2 = \{3\}$ is a valid partition.

Example

Next, we define the function which maps an instance of *Set-Partition* (A) to an instance of *Subset-Sum* (A, s). Suppose we let $\sigma_A = \sum_{x \in A} x$

Definition

$$r(A) = (A, \frac{\sigma_A}{2})$$

Then, we need to prove

Lemma

A is a yes-input of Set-Partition if and only if $r(A)$ is a yes-input of Subset-Sum

Example

Proof.

- ⇒ Assume A is a yes-input of Set-Partition. Then, A has a partition (A_1, A_2) such that $\sigma_{A_1} = \sigma_{A_2}$. By calculations, $\sigma_{A_1} = \sigma_A - \sigma_{A_2} = \sigma_A - \sigma_{A_1}$. Hence, $\sigma_{A_1} = \frac{\sigma_A}{2}$. Therefore, $A_1 \subset A$ is the subset whose sum is equal to $s = \frac{\sigma_A}{2}$. Therefore, $(A, \frac{\sigma_A}{2})$ is a yes-input of Subset-Sum.
- ⇐ Assume $r(A) = (A, \frac{\sigma_A}{2})$ is a yes-input of Subset-Sum. Then, there is a subset $B \subset A$ such that $\sigma_B = \frac{\sigma_A}{2}$. As for A , we construct a partition $(B, A \setminus B)$ such that $\sigma_{A \setminus B} = \sigma_A - \sigma_B = \sigma_A - \frac{\sigma_A}{2} = \frac{\sigma_A}{2} = \sigma_B$. Thus, $(B, A \setminus B)$ is a valid partition and A is a yes-input of Set-Partition.



Thus, the function r defined above is a reduction from Set-Partition to Subset-Sum.

Polynomial Time Reduction

Definition

A reduction r is a *polynomial time reduction* if r is a polynomial time computable function.

- If r is a polynomial time reduction from A to B , A is polynomial time reducible to B , denoted as $A \leq_p B$.
- Intuitively, problem B is no easier than problem A .
- The symbol " $\leq_p$ " shows the difficulty of the problems.
- The reduction from Set-Partition to Subset-Sum takes $O(n)$ time to calculate $\frac{\sigma_A}{2}$. Thus, it is a polynomial time reduction.
- Thus, $\text{Set-Partition} \leq_p \text{Subset-Sum}$.

NP-complete

Definition (Formal)

A problem P_1 is **NP-complete** if

- 1 $P_1 \in \mathbf{NP}$ and
- 2 $\forall P_2 \in \mathbf{NP}, P_2 \leq_p P_1$

Impossible to prove an **NP** problem A is **NP-complete** by enumerating all reductions, but

Theorem (Cook-Levin, 1971)

SAT is **NP-complete**.

After this, we only need to show $\mathbf{SAT} \leq_p A$ (or any **NP-complete** problem is polynomial time reducible to A).

Definition

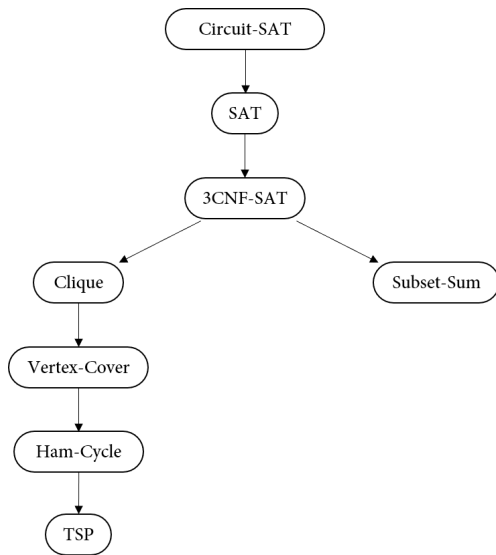
SAT: is the given boolean formula of n variables satisfiable?

Example,

- 4 boolean variables: x_1, x_2, x_3, x_4
- boolean formula: $\phi = ((x_1 \rightarrow x_2) \vee \neg((\neg x_1 \leftrightarrow x_3) \vee x_4)) \wedge \neg x_2$
- The output is “yes” because when $x_1 = 0, x_2 = 0, x_3 = 1, x_4 = 1$

$$\begin{aligned}\phi &= ((0 \rightarrow 0) \vee \neg((\neg 0 \leftrightarrow 1) \vee 1)) \wedge \neg 0 \\ &= (1 \vee \neg(1 \vee 1)) \wedge 1 \\ &= (1 \vee 0) \wedge 1 \\ &= 1\end{aligned}$$

Small picture



Let's try to prove a problem is **NP-complete**.

Assume we have known **Subset-Sum** is an **NP-complete** problem.

Set-Partition is a new problem to us. We want to show it is also in **NP-complete** using the reduction from **Subset-Sum**.

Polynomial Time Verifier

To show **Set-Partition** $\in$ **NP**, we give the polynomial verifier of each certificate.

Algorithm: **Set-Partition** verifier

Input: The set A , and a certificate (A'_1, A'_2)

```
1 begin
2   if  $(A'_1, A'_2)$  is not a partition of  $A$  then
3      $\quad$  return "No";
4   if  $\sum A'_1 \neq \sum A'_2$  then
5      $\quad$  return "No";
6   return "Yes";
```

Polynomial Time Reduction

To show $\text{Set-Partition} \in \mathbf{NP-c}$, we give the polynomial time reduction from Subset-Sum .

Definition

Let f be a function from the set of instances of Subset-Sum to the set of instances of Set-Partition such that

$$f(A, s) = A \cup \{\sigma_A - 2s\}$$

Lemma

f is a reduction from Subset-Sum to Set-Partition .

The proof is left to the students who are interested.

This reduction takes $O(n)$ time to calculate σ_A . Thus, it is a polynomial-time reduction.

Summary of **NP-c** proof

To prove a problem P is **NP-c**, you need to

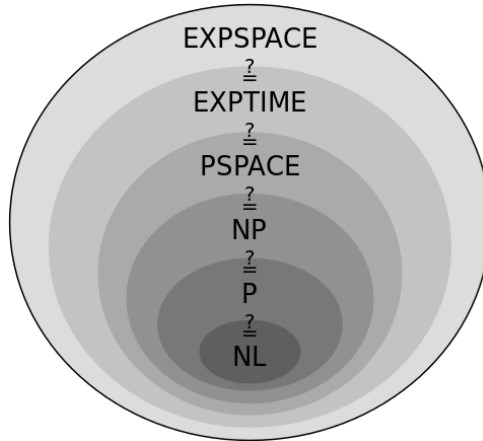
- show $P \in \mathbf{NP}$ by constructing a polynomial time verifier;
- choose a suitable **NP-c** problem P' , and show P' is polynomial time reducible to P .

Note that to show $P \in \mathbf{NP}$ is essential and the reduction is not from P to P' .

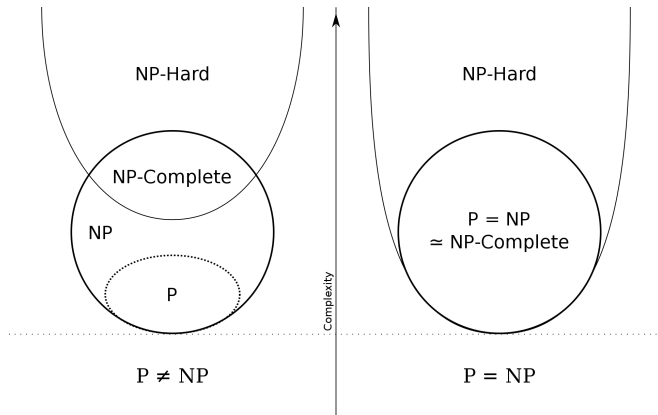
Some other complexity classes

- **NP-hard:** The class of optimization problems such that the corresponding decision problems are **NP-complete**.
- **co-NP:** The class of decision problems such that the no-inputs can be verified in polynomial time. (the complement of **NP**)
- **PSPACE:** The class of decision problems which can be solved in polynomial space.
- **EXPTIME:** The class of decision problems which can be solved in exponential time.
- **EXPSPACE:** The class of decision problems which can be solved in exponential space.

Complexity Zoo



Complexity Zoo



- Formulation of input size
- Decision problems and Optimization problems
- **P**, **NP**, and **NP-complete**
- Reduction and **NP-complete** proof
- Complexity Zoo